Analysis of Deep Learning Architectures for Real-Time Facial Recognition

Rahul Banyala (KU2407U434)
Prachi Ranjan (KU2407U433)
Rishu Pandey (KU2407U436)
Gauri Mahesh Adelkar (KU2407U401)

Abstract

This study deals with the real-time recognition of people using CNN-based deep learning models. It reviews and compares four different architectures: LightCNN-9, FaceNet, [Architecture 3], and [Architecture 4]. The comparison uses the experimental results and figures reported in the original research papers. Wherever possible, results obtained using the same facial dataset and evaluation protocol are selected to maintain a fair comparison. The models are compared based on recognition accuracy, computational requirements, inference speed, and suitability for real-time use. The study aims to identify which architecture provides the best balance between recognition performance and computational efficiency for a real-time facial recognition system. Later, the best-performing model will be used to build a real-time face recognition system using our own dataset.

I. Introduction

Facial recognition is a domain of computer vision that deals with recognising a human face and assigning a label or identity to it. It is commonly used in security systems, access control, attendance monitoring, surveillance, mobile-device authentication, and the organisation of digital images. Unlike face detection, which only finds the location of a face within an image, facial recognition attempts to determine the identity of the detected person. In a real-time facial recognition system, this complete process must be performed quickly enough to recognise people from a live image or video stream. Therefore, such a system requires not only good recognition accuracy but also efficient processing and a suitable response time.

The major challenge in facial recognition lies in the large variety of human faces and the different conditions in which facial images can be captured. Changes in the surrounding environment, lighting, facial position, camera angle, facial expression, and distance from the camera can significantly affect recognition performance. Other factors, such as low image resolution, image size, partial facial obstruction, background variation, ageing, and poor camera quality, can make the task even more difficult. The same person may appear different under different conditions, while two different people may sometimes have similar facial features. A reliable facial recognition model must therefore learn the important features that represent a person's identity while reducing the effect of these unwanted variations.

Within this field, Convolutional Neural Networks (CNNs) have become one of the major deep learning models used for recognising and classifying images. CNNs learn features directly from image data through hierarchical representations. The earlier layers generally learn simple features such as edges, lines, and textures, while the deeper layers learn more complex patterns such as facial structures and identity-related features. Their local connectivity and shared-weight mechanism allow them to process images efficiently and make them highly versatile for different image-related tasks. However, different CNN architectures may produce different results in terms of recognition accuracy, model size, computational requirements, and processing speed.

In this study, four CNN-based architectures are reviewed and compared for facial recognition: LightCNN-9, FaceNet, [Architecture 3], and [Architecture 4]. LightCNN-9 is a compact facial recognition architecture that uses the Max-Feature-Map activation to select more informative features and reduce the effect of less useful information. Its relatively lightweight structure makes it an important candidate for a real-time facial recognition system

where computational efficiency is required. The remaining architectures are also studied to understand how their designs influence recognition performance, efficiency, and suitability for real-time use.

The comparison is based on the experimental results and figures reported in the original research papers. To make the comparison as fair as possible, results obtained using similar datasets, evaluation protocols, preprocessing methods, and performance metrics are selected wherever they are available. Results from different testing conditions are not treated as directly equivalent, and these differences are considered while discussing the performance of each architecture. The models are compared using factors such as recognition accuracy, inference speed, model size, and computational requirements.

The main purpose of this study is to identify the architecture that provides the best balance between recognition performance and computational efficiency. The best-performing and most suitable model will later be selected to build a real-time facial recognition system trained on our own dataset. This comparison will therefore provide the foundation for choosing a model that can recognise the required individuals accurately while still being practical for real-time implementation.

II. Architecture 1: LightCNN-9

LightCNN was introduced by Wu et al. to learn facial features from large-scale datasets containing incorrectly labelled and noisy images [1]. The main objective of this architecture is to achieve reliable facial recognition while reducing the model size and computational requirements. LightCNN-9 is one of the smaller architectures proposed in the original study, making it suitable for systems where recognition speed and available hardware are important.

The main feature of LightCNN is the Max-Feature-Map (MFM) operation, which is used instead of a conventional activation function such as ReLU. A convolutional layer first produces $2N$ feature maps, which are divided into pairs. MFM compares the corresponding values in each pair and keeps the maximum value:

$$\hat{x}_{ij}^{k} = \max\left(x_{ij}^{k}, x_{ij}^{k+N}\right) \tag{1}$$

As a result, $2N$ input feature maps are reduced to N output feature maps. This creates competition between the feature maps and allows the network to retain the stronger facial information while suppressing weaker or noisy responses.

LightCNN-9 takes a 128×128 grayscale facial image as its input. The first stage contains a 5×5 convolution followed by MFM and max pooling. It is followed by four convolution blocks, with each block containing a 1×1 convolution, MFM, a 3×3 convolution, and another MFM operation. Four max-pooling operations gradually reduce the spatial dimensions to $8 \times 8 \times 128$. The resulting features are flattened into 8,192 values and passed through a fully connected layer containing 512 outputs. MFM then reduces this output to a compact 256-dimensional facial representation. Finally, a fully connected classifier produces a score for each identity class. The complete flow of the architecture is presented in Figure 1.

According to the original paper, LightCNN-9 contains approximately 5.556 million parameters and requires approximately 1 GFLOP. It produces a 256-dimensional facial representation and requires approximately 67 ms for feature extraction on a single core of an Intel i7-4790 processor. The paper reports 98.80% accuracy on LFW and 93.40% accuracy on YTF under its stated evaluation protocols [1].

The smaller model size, low-dimensional representation, and use of grayscale images make LightCNN-9 a suitable candidate for real-time facial recognition. However, its fully connected layer requires a fixed input size, and the reported processing time depends on the hardware used by the original authors. The LFW result also represents face verification rather than the identity-classification task required by our final system. For our implementation, the final classifier can be replaced with a new classifier for the identities in our own dataset while retaining the earlier layers for facial feature extraction.

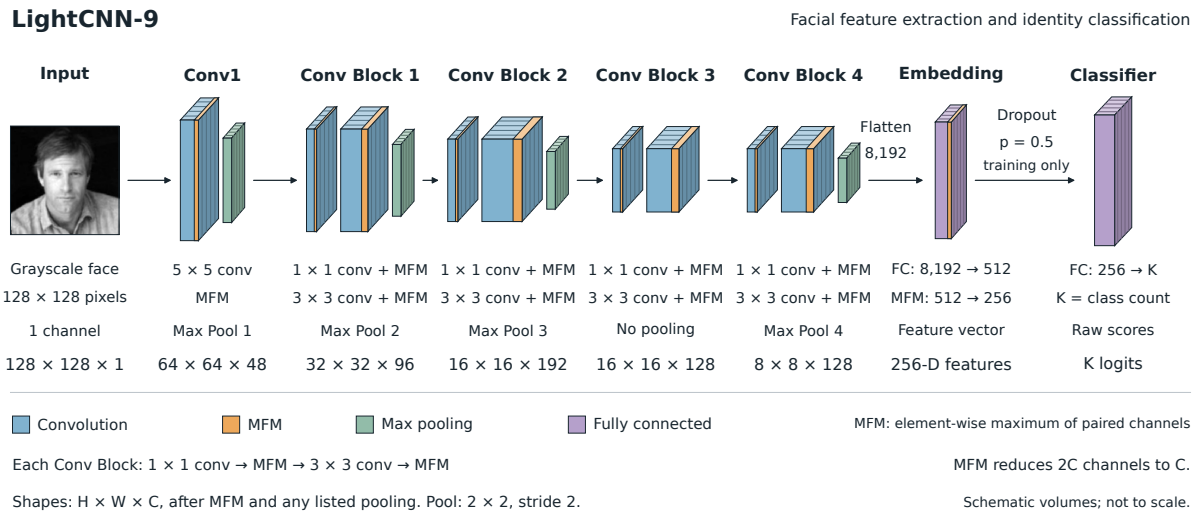


Figure 1: *Architecture of LightCNN-9 showing the convolution blocks, MFM operations, pooling layers, 256-dimensional embedding, and identity classifier.*

III. Architecture 2: FaceNet

FaceNet is a unified system for face verification, recognition, and clustering, which builds upon the Siamese Neural Network (SNN) architecture. Each face is mapped onto the Euclidean space. The network is trained such that the embeddings of similar data points converge closer together within the feature space, while simultaneously pushing embeddings of dissimilar data points to drift further apart; meaning images of the same person are close together in the learned space and vice versa in the case of images of different people [2].

It also uses a triplet loss function. To calculate triplet loss, we use images corresponding to the anchor, positive, and negative.

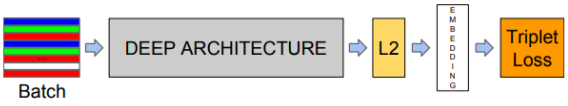


Figure 2: *Model Structure of FaceNet*

The network consists of an input layer, a deep CNN, followed by L2 normalization, which results in the face embedding. This is then followed by the triplet loss.

Triplet Loss and Selection

The loss function is defined using triplets of training points of the form (A, P, N).

A = anchor (reference image)

P = positive (if the image is of the same person as A)

N = negative (if the image is of a different person than A)

Triplet loss minimizes the distance between the anchor and the positive, while maximizing the distance between the anchor and the negative (see Figure 3).

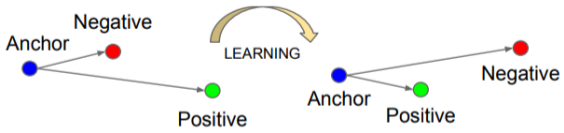


Figure 3: *Triplet Loss*

Triplet loss can be formalized as:

$$L = \sum_{i}^{N} \Big[\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha \Big]_+ \tag{2}$$

α here is defined as the margin that is enforced between positive and negative pairs.

Generating all possible triplets would result in multiple triplets that already satisfy the constraint. These triplets would not contribute to the training and would result in slower convergence. And thus it is important to select hard triplets which can therefore contribute to improving the model.

Deep Convolution Networks

FaceNet trains CNNs using Stochastic Gradient Descent (SGD) with standard backdrop and AdaGrad.

SDG is an optimization model that is used to optimize the loss function

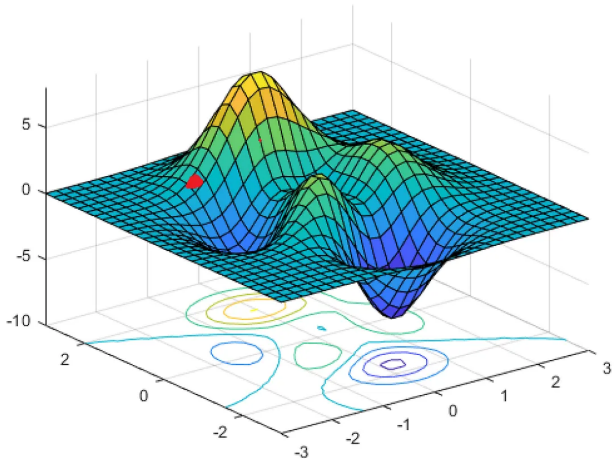


Figure 4: *SDG (Credit - datascience.stackexchange)*

The x-axis and y-axis represent the weights while the z-axis is the loss corresponding to those weights. The intuition behind SDG is to traverse the hill-like structure using a point in such a way that the global minima can be obtained.

In SGD, the gradient is a vector which indicates the direction of the steepest local increase. SGD then updates the parameter in the opposite direction to minimize the loss, using gradients estimated from randomly sampled training data.

While AdaGrad is utilized here to dynamically generate variable learning rates, as fixed learning rates are impractical in deep learning.

ReLU (Rectified Linear Unit)

ReLU is a non-linear activation function. Without non-linear activation functions, even multiple layers of convolution would collapse into a single linear transformation. Also, without activation functions, complex and intricate patterns cannot be learned effectively, as the decision boundary would remain linear.

ReLU is a widely used alternative to traditional activation functions such as sigmoid and tanh. This activation function helps mitigate the vanishing gradient problem, as its gradient remains constant for positive inputs.

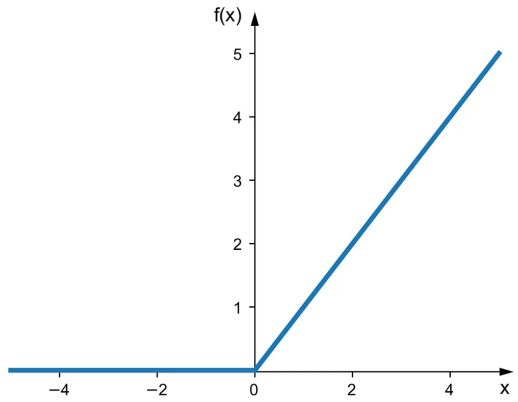


Figure 5: *ReLU activation function*

CNN Architectures

The two types of CNN architectures used in FaceNet are Zeiler & Fergus architecture and Inception Model.

Zeiler & Fergus architecture - This architecture introduced a novel visualization technique that gives insight into the function of intermediate layers and the operation of classifiers in a CNN.

layer	size-in	size-out	kernel	param	FLPS
conv1	220×220×3	110×110×64	7×7×3, 2	9K	115M
pool1	110×110×64	55×55×64	3×3×64, 2	0	
rnorm1	55×55×64	55×55×64		0	
conv2a	55×55×64	55×55×64	1×1×64, 1	4K	13M
conv2	55×55×64	55×55×192	3×3×64, 1	111K	335M
rnorm2	55×55×192	55×55×192		0	
pool2	55×55×192	28×28×192	3×3×192, 2	0	
conv3a	28×28×192	28×28×192	1×1×192, 1	37K	29M
conv3	28×28×192	28×28×384	3×3×192, 1	664K	521M
pool3	28×28×384	14×14×384	3×3×384, 2	0	
conv4a	14×14×384	14×14×384	1×1×384, 1	148K	29M
conv4	14×14×384	14×14×256	3×3×384, 1	885K	173M
conv5a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv5	14×14×256	14×14×256	3×3×256, 1	590K	116M
conv6a	14×14×256	14×14×256	1×1×256, 1	66K	13M
conv6	14×14×256	14×14×256	3×3×256, 1	590K	116M
pool4	14×14×256	7×7×256	3×3×256, 2	0	
concat	7×7×256	7×7×256		0	
fc1	7×7×256	1×32×128	maxout p=2	103M	103M
fc2	1×32×128	1×32×128	maxout p=2	34M	34M
fc7128	1×32×128	1×1×128		524K	0.5M
L2	1×1×128	1×1×128		0	
total				140M	1.6B

Figure 6: *Zeiler & Fergus architecture used in FaceNet*

Inception Model - The architecture was first introduced by the researchers at Google to curb the exorbitant resource requirement of deep neural networks. The Inception architecture combines 1x1, 3x3 and 5x5 Convulation filters in parallel and extract the features from the input image at different scales. The output of each filter is concatenated and passed to the next layer. This process is then repeated. This approach helps capture the local

and global features while also minimizing the number of parameters required.

type	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj (p)	params	FLOPS
conv1 (7×7×3, 2)	112×112×64	1							9K	119M
max pool + norm	56×56×64	0						m 3×3, 2		
inception (2)	56×56×192	2		64	192				115K	360M
norm + max pool	28×28×192	0						m 3×3, 2		
inception (3a)	28×28×256	2	64	96	128	16	32	m, 32p	164K	128M
inception (3b)	28×28×320	2	64	96	128	32	64	L_2, 64p	228K	179M
inception (3c)	14×14×640	2	0	128	256,2	32	64,2	m 3×3,2	398K	108M
inception (4a)	14×14×640	2	256	96	192	32	64	L_2, 128p	545K	107M
inception (4b)	14×14×640	2	224	112	224	32	64	L_2, 128p	595K	117M
inception (4c)	14×14×640	2	192	128	256	32	64	L_2, 128p	654K	128M
inception (4d)	14×14×640	2	160	144	288	32	64	L_2, 128p	722K	142M
inception (4e)	7×7×1024	2	0	160	256,2	64	128,2	m 3×3,2	717K	56M
inception (5a)	7×7×1024	2	384	192	384	48	128	L_2, 128p	1.6M	78M
inception (5b)	7×7×1024	2	384	192	384	48	128	m, 128p	1.6M	78M
avg pool	1×1×1024	0								
fully conn	1×1×128	1							131K	0.1M
L2 normalization	1×1×128	0								
total									7.5M	1.6B

Figure 7: *Inception Model used in FaceNet*

Performance of FaceNet Model on LFW

The model is evaluated on two modes:

Fixed center crop of the LFW provided thumbnail.

A face detector is run on the LFW thumbnails. If it fails to align the images, the LFW alignment is used.

FaceNet achieves a classification accuracy of **98.87%** $\pm$ **0.15** when using the fixed center crop and **99.65%** $\pm$ **0.09** standard error of the mean when using the extra face alignment.



Figure 8: *LFW errors. This shows all pairs of images that were incorrectly classified on LFW*

Conclusion

With an accuracy of **99.65%** $\pm$ **0.09** on the LFW dataset, FaceNet demonstrates the effectiveness of embedding-based approaches in face recognition.

IV. Architecture 3: Name

[Write the analysis of Architecture 3 here.]

V. Architecture 4: Name

[Write the analysis of Architecture 4 here.]

VI. Analysis and Discussion

The four architectures will be compared using the results reported in their original research papers. LFW is selected as the primary dataset because it is widely used for evaluating facial recognition models. YTF is selected as the second dataset because it evaluates recognition from video frames containing changes in pose, expression, lighting, and image quality. A result will only be treated as a direct comparison when the models use the same dataset, evaluation protocol, and performance metric. Results obtained using different protocols will be discussed separately. Along with recognition performance, the comparison will consider the number of parameters, embedding dimension, inference time, and the hardware used for testing. Table 1 provides the common structure for recording these results. The remaining values will be added after the other architectures have been studied.

VII. Conclusion

[Write the approximately 100 word conclusion here.]

References

[1] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 11, pp. 2884–2896, 2018.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 815–823, 2015.

Table 1: *Preliminary comparison of the four facial recognition architectures using results reported in their original papers.*

Architecture	LFW Acc. (%)	YTF Acc. (%)	Parameters	Embedding	Inference time
LightCNN-9	98.80	93.40	5.556M	256-D	67 ms[a]
FaceNet	99.65[b]	95.12	7.5M[c]	128-D	–[d]
Architecture 3	–	–	–	–	–
Architecture 4	–	–	–	–	–

[a] *Reported for feature extraction using one core of an Intel i7-4790 processor.* [b] *The supplied FaceNet document reports 99.65% ± 0.09 using extra face alignment.* [c] *The Inception model table reports 7.5M parameters and 1.6B FLOPS.* [d] *Inference time and tested hardware are not reported. Accuracy values should only be compared directly when the evaluation protocols are equivalent.*